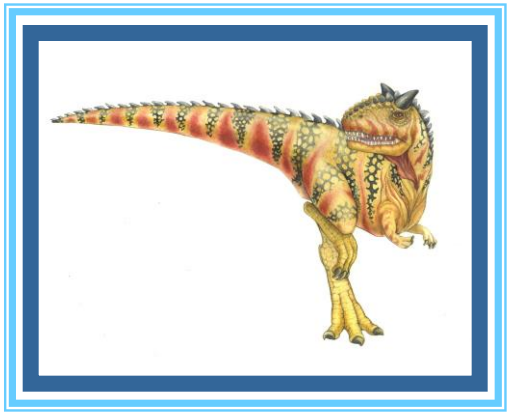
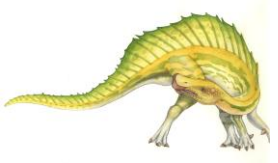
Chapter 4: Threads & Concurrency



Implementarea threadurilor kernel

- pachetele de threaduri se pot implementa in kernel sau in spatiul utilizator
- threadurile kernel separa campurile din PCB care ajuta la crearea unui punct de executie si le stocheaza intr-un TCB
- astfel, un proces cu un singur punct de executie e reprezentat in kernel de un PCB si un TCB
- operatia de creare a unui thread (*thread_create*) este un apel sistem
  - aloca un TCB
  - aloca stive kernel si user
  - le leaga la PCB-ul procesului in care s-a facut apelul sistem



Exemplu cu doua threaduri kernel

- un thread suspendat in kernel
- altul ruland in spatiul utilizator
- pt ca sunt implementate in kernel si seamana cu procesele, threadurile kernel se mai cheama si *procese usoare* (*lightweight processes, LWP*)

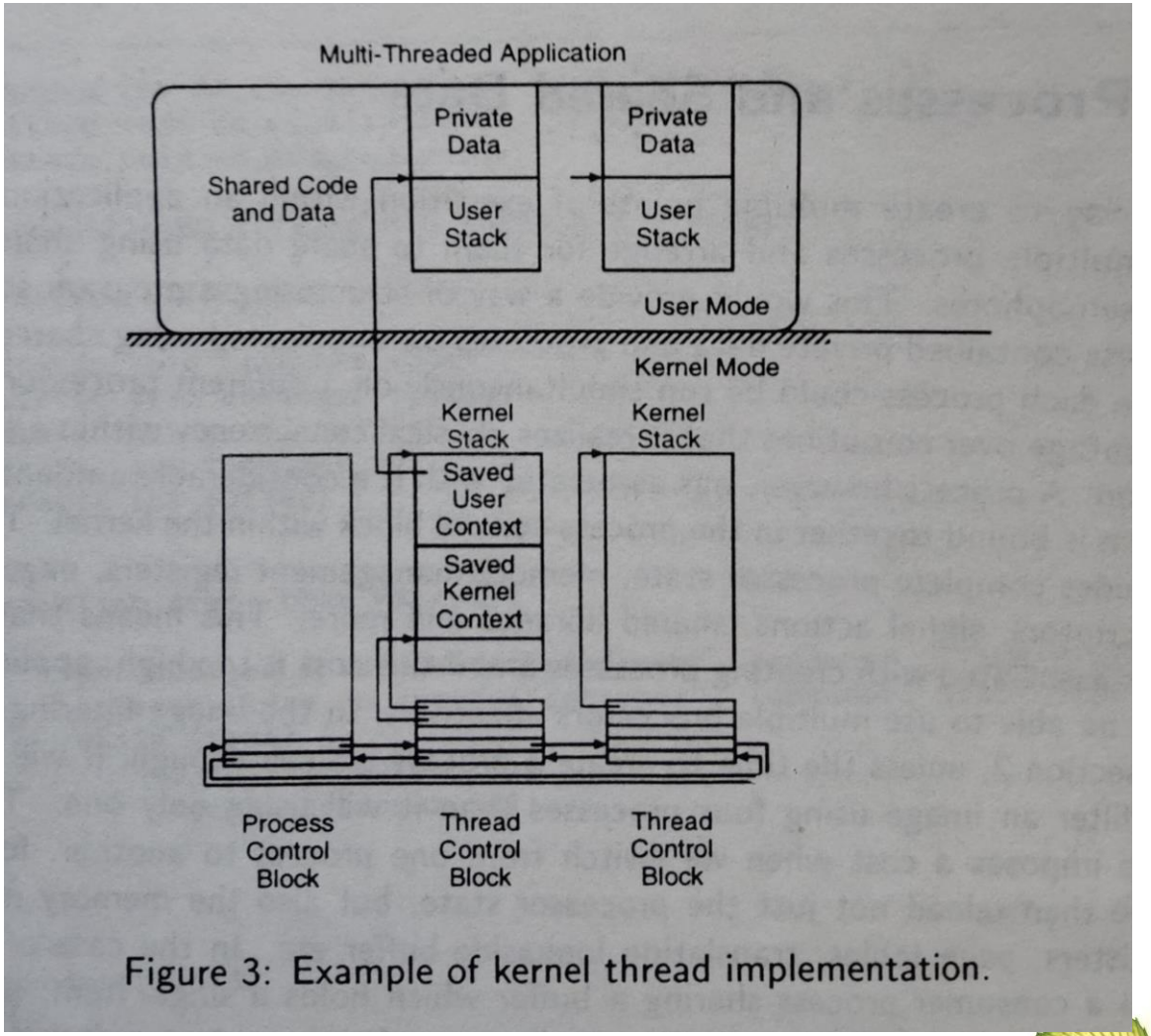


Figure 3: Example of kernel thread implementation.

Costuri threaduri kernel

- costul crearii unui thread kernel << costul crearii unui proces
  - se aloca si initializeaza doar TCB-ul si stivele
  - restul contextului exista deja creat in PCB
- context switch
  - intre doua threaduri **ale aceluiasi proces** << schimbarea contextului a doua procese (nu trebuie schimbat restul contextului din PCB)
  - intre doua threaduri din procese **diferite** are acelasi cost ca si context switch-ul de procese (nu se schimba doar TCB-urile, ci si PCB-urile)

Planificarea threadurilor kernel

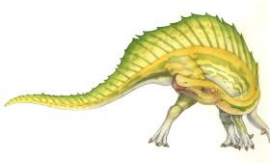
- threadurile ruleaza asincron unele fata de celelalte si pot pierde procesorul la fel ca si procesele

  => acces sincronizat date partajate pt. IPC

- planificatorul kernel alege urmatorul thread care trebuie sa ruleze

  => politicile de planificare user (de ex bazate pe prioritati) trebuie comunicate kernelului

- *gang scheduling*
  - in cazul multiprocesoarelor, planificatorul poate asigna mai multe CPU-uri unui singur proces pt. ca threadurile sa ruleze in paralel
- thread blocat in kernel (de ex prin apel de sistem I/O)
  - pp cuanta de timp alocata procesului nu a expirat
  - planificatorul cauta in lista de TCB-uri un thread gata de rulare **din acelasi proces**
  - planifica thread-ul pt rulare



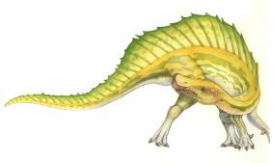
Protectia threadurilor kernel

- observatia generala despre threaduri e valabila si pt threaduri kernel
- un thread poate corupe stiva altui thread, de pilda => se distrug datele private ale altui thread (variabilele automatice/locale)
- solutie: implementarea stivelor in spatii de adresa diferite, dar asta mareste costul context switch-ului
  - mai exact, ar fi nevoie de salvarea si reincarcarea contextelor de executie referitoare la gestiunea memoriei, pe langa contextul uzual din TCB

Dezavantajele threadurilor kernel

- mai putin costisitoare ca procesele, dar anumite aspecte sunt nepotrivite pt utilizatori
- (1) *thread_create* este apel sistem => costisitor pt procese care creeaza multe threaduri
- (2) context switching-ul de threaduri => intrarea si iesirea in/din kernel mode => overhead aditional context switching-ului obisnuit
- (3) implementarea in kernel **inflexibila**
  - impune un model de threaduri care nu e potrivit pt orice aplicatie
  - codul planificatorului (scheduler) nu e accesibil (fiind in kernel)

  => greu de adaptat pt cerintele specifice ale unei anumite aplicatii (politica de planificare nu se poate schimba usor)



User-level threads

- daca planificatorul si TCB-urile sunt implementate in spatiul utilizator costurile scad (nu mai e nevoie de transgresarea granitei kernel/user)
- comparatie calitativa, intr-un ex in care un apel de procedura cost 7 usec, iar un apel sistem (trap) 19 usec

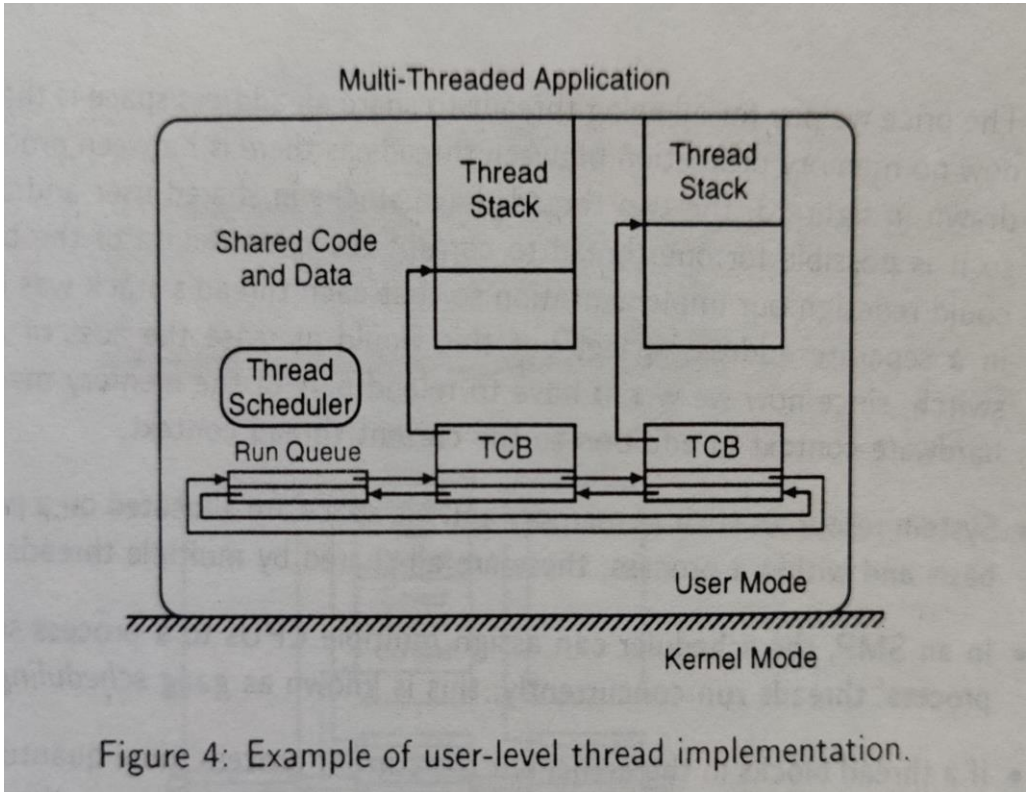
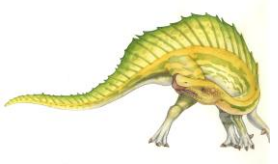


Figure 4: Example of user-level thread implementation.

Operatie	Thread user	Thread kernel	Proces Unix
fork	34 usec	948 usec	11300 usec
IPC synch	37 usec	441 usec	1840 usec

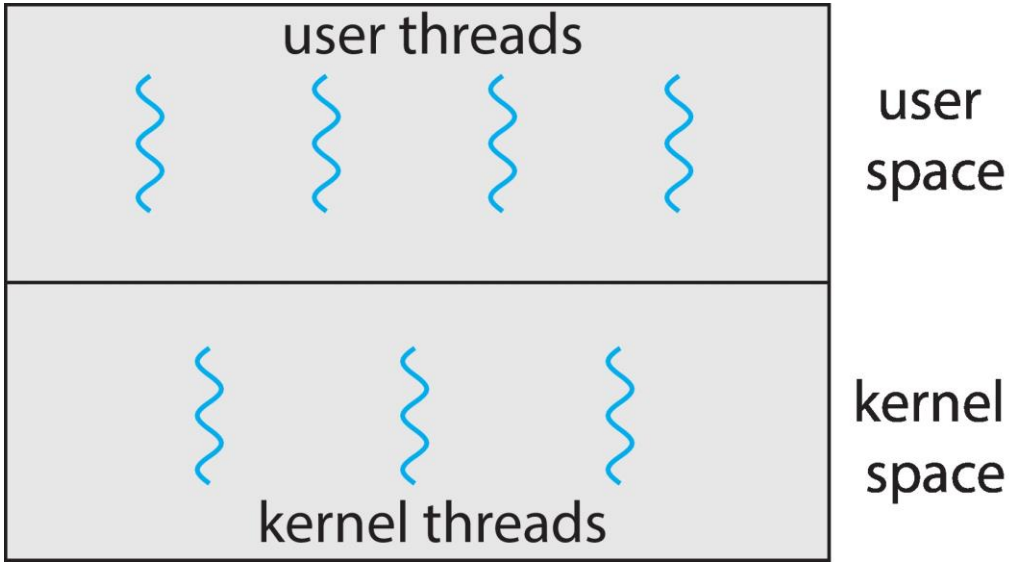


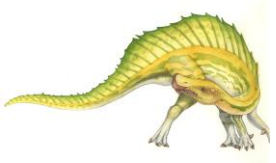
Caracteristici threaduri utilizator

- creare si context switch
  - rapide, nu se trece granita user-kernel
  - kernelul nu trebuie sa verifice parametrii apelului sistem
- aplicatiile pot furniza propriul planificator (*thread scheduler*)
  - customizat cf unor cerinte specifice
- nu necesita suport explicit din partea kernelului; procesul e privit ca un procesor virtual pt. threaduri
- fiind mult mai rapide decat threadurile kernel, de regula threadurile user se implementeaza deasupra threadurilor kernel

  => planificatorul de threaduri user trateaza threadurile kernel ca pe procesoare virtuale si multiplexeaza mai multe threaduri user pe unul sau mai multe threaduri kernel

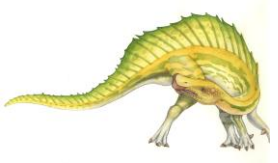
User and Kernel Threads





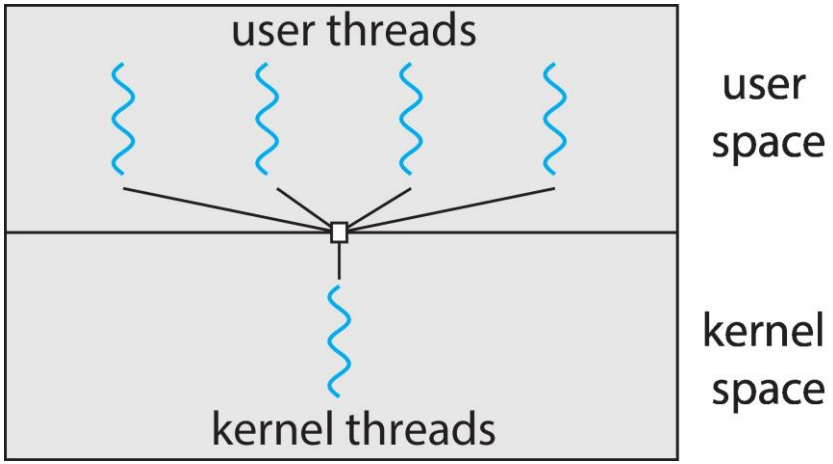
Multithreading Models

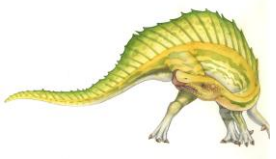
- Many-to-One
- One-to-One
- Many-to-Many



Many-to-One

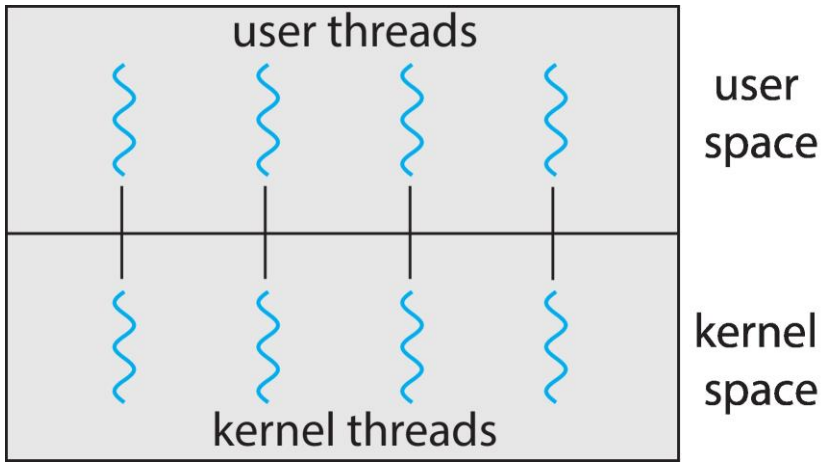
- Many user-level threads mapped to single kernel thread
- One thread blocking causes all to block
- Multiple threads may not run in parallel on multicore system because only one may be in kernel at a time
- Few systems currently use this model
- Examples:
  - **Solaris Green Threads**
  - **GNU Portable Threads**





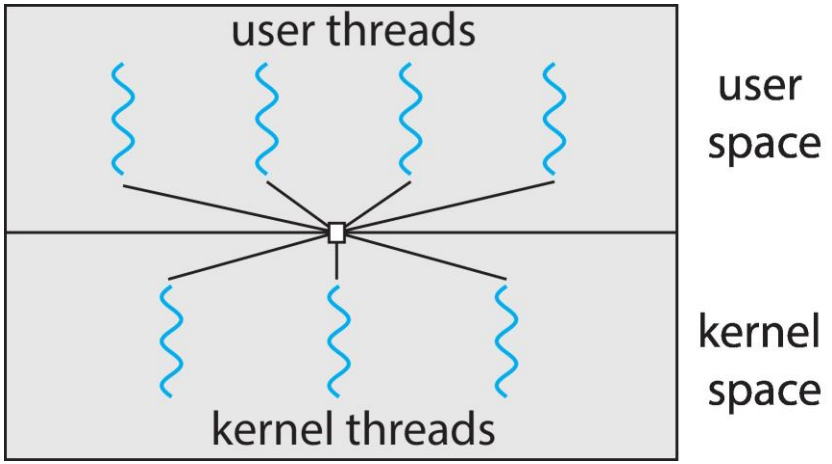
One-to-One

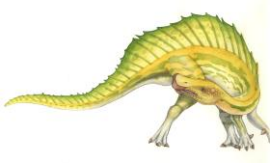
- Each user-level thread maps to kernel thread
- Creating a user-level thread creates a kernel thread
- More concurrency than many-to-one
- Number of threads per process sometimes restricted due to overhead
- Examples
  - Windows
  - Linux



Many-to-Many Model

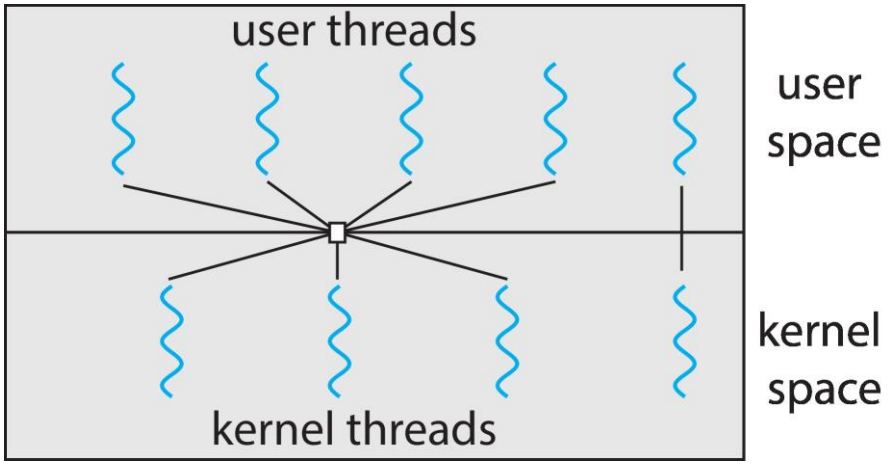
- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- Windows with the *ThreadFiber* package
- Otherwise not very common

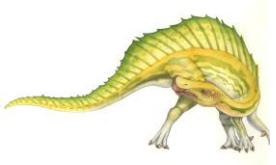




Two-level Model

- Similar to M:M, except that it allows a user thread to be **bound** to kernel thread



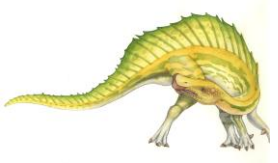


Probleme comune threaduri kernel/user

- *reentranta*: multe apeluri de biblioteca sunt ne-reentrante (apelurile reentrante desemnate in paginile de manual ca fiind *thread safe*)
  - ex: trimiterea unui mesaj in retea in 2 pasi: (a) asamblare mesaj in buffer + (b) transmisie (*syscall*)
  - pierderea procesorului intre (a) si (b) poate conduce la suprascrierea bufferului
  - alte ex: errno, malloc, apeluri stdio
- tratarea semnalelor
  - ex: un thread trateaza un semnal in vreme ce alt thread vrea ca semnalul respectiv sa termine aplicatia (procesul, mai exact)
  - se poate intampla daca se folosesc apeluri de biblioteca si runtime user impreuna
  - problema deriva din faptul ca semnalele sunt definite per proces si nu per thread

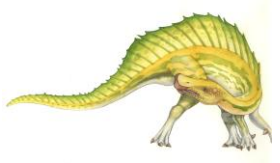
Dezavantaje threaduri utilizator

- determinate de faptul ca existenta lor e necunoscuta kernelului
- (1) thread user executa apel sistem blocant in kernel
  - planificatorul kernel considera intreg procesul blocat (nu are cunostinta despre thread-urile user)
  - aloca CPU altui proces (sau kernel thread), *chiar daca procesul curent mai are si alte threaduri user gata de rulare si nu si-a consumat toata cuanta de timp CPU !*
- (2) thread user comite *page fault* (acces la pagina de memorie inexistenta/nealocata)
  - planificatorul alege alt procesor/kernel thread in vreme ce pagina de memorie e adusa de pe disc => aplicatia ruleaza cu mai putine CPU decat e necesar



Dezavantaje threaduri utilizator (cont.)

- (3) kernelul poate lua procesorul unui thread user care detine un *spinlock*
  - kernelul nu e constient de existenta threadurilor user
  - consecinta: scadere dramatica de performanta pt aplicatiile care folosesc threaduri user in paralel
  - efectul agravat daca schedulerul alege sa ruleze alte threaduri care vor sa obtina acelasi *spinlock*
- problema de fond: lipsa de coordonare intre schedulerul kernel si implementarea pachetului de threaduri user



Scheduler activations

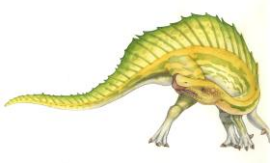
- metoda de coordonare kernel – pachet de threaduri utilizator
- model: aplicatia ruleaza pe un multiprocesor virtual
  - exista apeluri sistem pt a suplimenta/diminua nr de procesoare virtuale alocate de kernel (“adauga CPU”, “acest CPU este idle”)
  - kernelul decide daca onoreaza cererea sau nu
- scheduler activation e aproximarea unui kernel thread
  - are stive kernel si user
  - ofera context de executie pt un thread user
  - in plus, ofera conceptul de *upcall* pt evenimente de mai multe tipuri
  - fiecare *upcall* creeaza o noua activare (optimizare: folosirea vechilor activari)

Tipuri de evenimente upcall

- *adauga procesor* – consecinta este executia unui thread user
- *procesor preemptat* – adauga threadul user care se executa in activarea care a pierdut CPU in coada de threaduri gata de rulare
- *activare blocata* – activarea s-a blocat (eg, apel sistem blocant) si nu mai utilizeaza procesorul
- *activare deblocata* – pune in lista gata de rulare threadul care se executa in contextul activarii blocate

Functionarea scheduler activations

- la pornirea procesului
  - kernelul aloca o activare + notifica aplicatia (*upcall* “adauga CPU”) dupa ce i-a asigurat un CPU
  - sistemul de gestiune al threadurilor user primeste notificarea
  - noua activare devine context de executie pt initializarea sistemului de gestiune a thread-urilor si a threadului *main*
    - ulterior se pot crea noi threaduri si cere noi procesoare
- la o cerere de suplimentare a concurentei (adaugare CPU sau pt. un thread care se blocheaza)
  - kernelul salveaza starea threadului user in activarea curenta
  - aloca o noua activare
  - cheama aplicatia in contextul noii activari

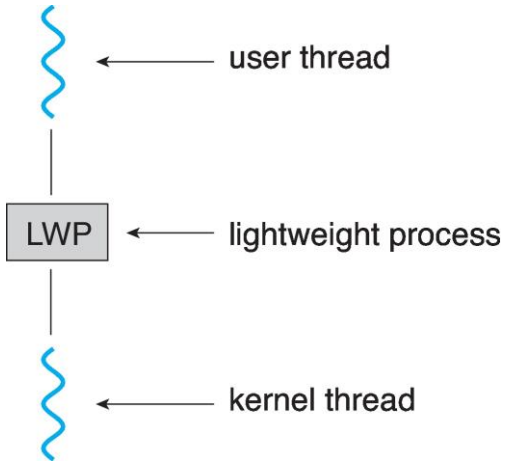


Functionarea scheduler activations (cont)

- la blocarea unui thread user in kernel
  - nu se continua in kernel la deblocare
  - se creeaza o noua activare
  - se notifica aplicatia
  - schedulerul **user** copiaza starea threadului blocat din vechea activare si notifica kernelul ca vechea activare poate fi refolosita
- cand un thread user care detine un spinlock pierde procesorul
  - kernelul genereaza un *upcall*
  - sistemul de gestiune a threadurilor verifica daca threadul e in sectiune critica
  - in caz afirmativ, threadul este continuat temporar printr-un context switch in user space
    - la iesirea din sectiunea critica, se da controlul inapoi *upcall-ului* tot prin context switch in user space

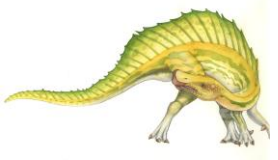
Scheduler Activations

- Both M:M and Two-level models require communication to maintain the appropriate number of kernel threads allocated to the application
- Typically use an intermediate data structure between user and kernel threads – **lightweight process** (**LWP**)
  - Appears to be a virtual processor on which process can schedule user thread to run
  - Each LWP attached to kernel thread
  - How many LWPs to create?
- Scheduler activations provide **upcalls** - a communication mechanism from the kernel to the **upcall handler** in the thread library
- This communication allows an application to maintain the correct number kernel threads



Thread Libraries

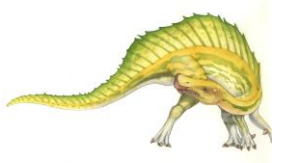
- **Thread library** provides programmer with API for creating and managing threads
- Two primary ways of implementing
  - Library entirely in user space
  - Kernel-level library supported by the OS



Exemple de pachete de threaduri user

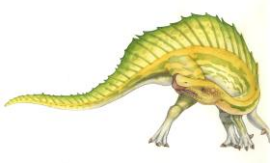
- Solaris si POSIX
- threaduri Solaris
  - kernel (s.n. LWPs, multiplexate pe CPU existente) sau utilizator
  - pachetul de threaduri user (*libthread*) planifica threadurile user pe o colectie de LWPs
  - two-level model
    - uzual, threadurile user sunt create nelegate (*unbound*), se pot muta intre LWP-uri
    - un thread user asignat unui LWP e numit legat (*bound*)
  - controlul kernelului asupra threadurilor user se exercita prin bound threads cu ajutorul setarii prioritatii LWP-ului asociat

    ex: thread user pt stream audio cu prioritate mare in aplicatie multimedia



Exemple de pachete de threaduri user

- threaduri Solaris
- ….
  - biblioteca de threaduri asigura ca exista suficiente LWP-uri active pt ca procesul sa poata continua
  - daca un proces determina toate LWP-urile sale sa se blocheze, biblioteca va crea LWP-uri aditionale pt. ca threadurile neblocate sa poata rula
  - daca LWP-urile sunt *idle* prea mult timp, biblioteca le da inapoi kernelului pt a fi folosite de alte aplicatii



Exemple de pachete de threaduri user

- exista apel de setare explicita a nr de LWP-uri la un nou nivel

  thr_setconcurrency(int new_level)
- mecanisme IPC: *mutex* si *variabile conditie*
- implementeaza TLS (Thread-Local Storage), capacitate privata de stocare a variabilelor in afara stivei
  - variabile locale nepartajate, “statice” (globale pt threadul respectiv)
  - declarate cu #pragma, de ex #pragma unshared errno;
- threaduri POSIX (pthreads)
  - standard IEEE, Portable Operating System Interface
    - specificatie, nu implementare
  - asemanatoare threadurilor Solaris

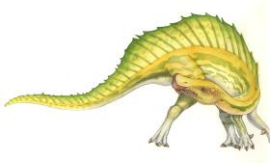
Solaris threads

```
int thr_create(void *stack_base, size_t stack_size,
               void *(*function)(void *), void *arg, long flags)
void thr_exit(void *status)
int thr_join(thread_t wait_for, thread_t *joiner_id, void *status)
int mutex_init(mutex_t *mutex, int type, void *arg)
int mutex_lock(mutex_t *mutex)
int mtex_unlock(mutex_t *mutex);
int cond_init(cond_t *cond, int type, int arg)
int cond_wait(cond_t *cond, mutex_t *mutex)
int cond_signal(cond_t *cond)
int cond_broadcast(cond_t *cond)
```

Figure 5: Some commonly used UI thread operations.

Pthreads

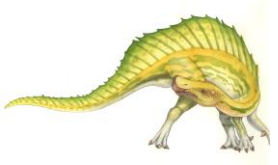
- May be provided either as user-level or kernel-level
- A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization
- ***Specification***, not ***implementation***
- API specifies behavior of the thread library, implementation is up to development of the library
- Common in UNIX operating systems (Linux & Mac OS X)



POSIX threads

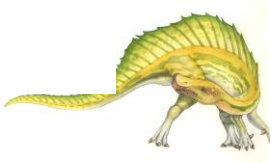
```
int pthread_create(pthread_t *thread_pointer,
                   pthread_attr_t *attributes,
                    void *(*function)(void *), void *arg)
void pthread_exit(void *status)
int pthread_join(pthread_t thread, void **status)
void pthread_mutex_init(pthread_mutex_t *mutex,
                        pthread_mutexattr_t *attr)
int pthread_mutex_lock(pthread_mutex_t *mutex)
int pthread_mutex_unlock(pthread_mutex_t *mutex)
int pthread_cond_init(pthread_cond_t *cond, pthread_condattr_t *attr)
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex)
int pthread_cond_signal(pthread_cond_t *cond)
int pthread_cond_broadcast(pthread_cond_t *cond)
```

Figure 6: Some commonly used Pthreads operations.



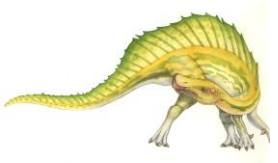
Pthreads Example

```
#include <pthread.h>
#include <stdio.h>

#include <stdlib.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
   pthread_t tid; /* the thread identifier */
   pthread_attr_t attr; /* set of thread attributes */

   /* set the default attributes of the thread */
   pthread_attr_init(&attr);
   /* create the thread */
   pthread_create(&tid, &attr, runner, argv[1]);
   /* wait for the thread to exit */
   pthread_join(tid,NULL);

   printf("sum = %d\n",sum);
}
```



Pthreads Example (Cont.)

```
/* The thread will execute in this function */
void *runner(void *param)
{
   int i, upper = atoi(param);
   sum = 0;

   for (i = 1; i <= upper; i++)
      sum += i;

   pthread_exit(0);
}
```



Pthreads Code for Joining 10 Threads

```
#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
   pthread_join(workers[i], NULL);
```

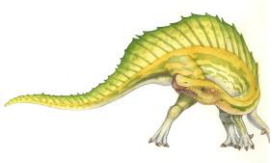
Implicit Threading

- Growing in popularity as numbers of threads increase, program correctness more difficult with explicit threads
- Creation and management of threads done by compilers and run-time libraries rather than programmers
- Five methods explored
  - Thread Pools
  - Fork-Join
  - OpenMP
  - Grand Central Dispatch
  - Intel Threading Building Blocks

Thread Pools

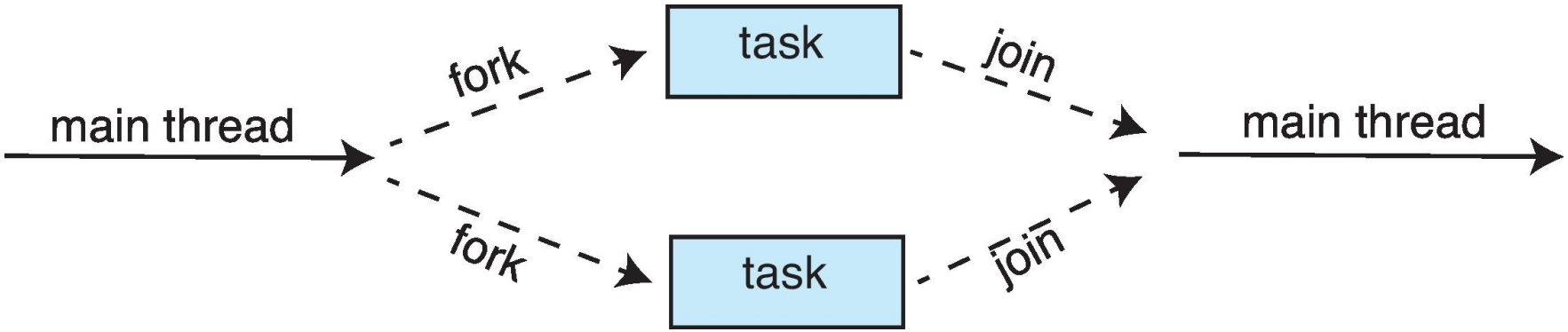
- Create a number of threads in a pool where they await work
- Advantages:
  - Usually slightly faster to service a request with an existing thread than create a new thread
  - Allows the number of threads in the application(s) to be bound to the size of the pool
  - Separating task to be performed from mechanics of creating task allows different strategies for running task
    - i.e,Tasks could be scheduled to run periodically
- Windows API supports thread pools:

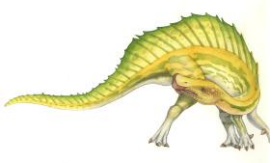
```
DWORD WINAPI PoolFunction(AVOID Param) {
    /*
    * this function runs as a separate thread.
    */
}
```



Fork-Join Parallelism

- Multiple threads (tasks) are **forked**, and then **joined**.

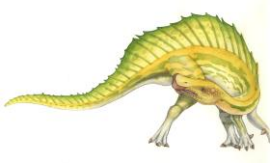




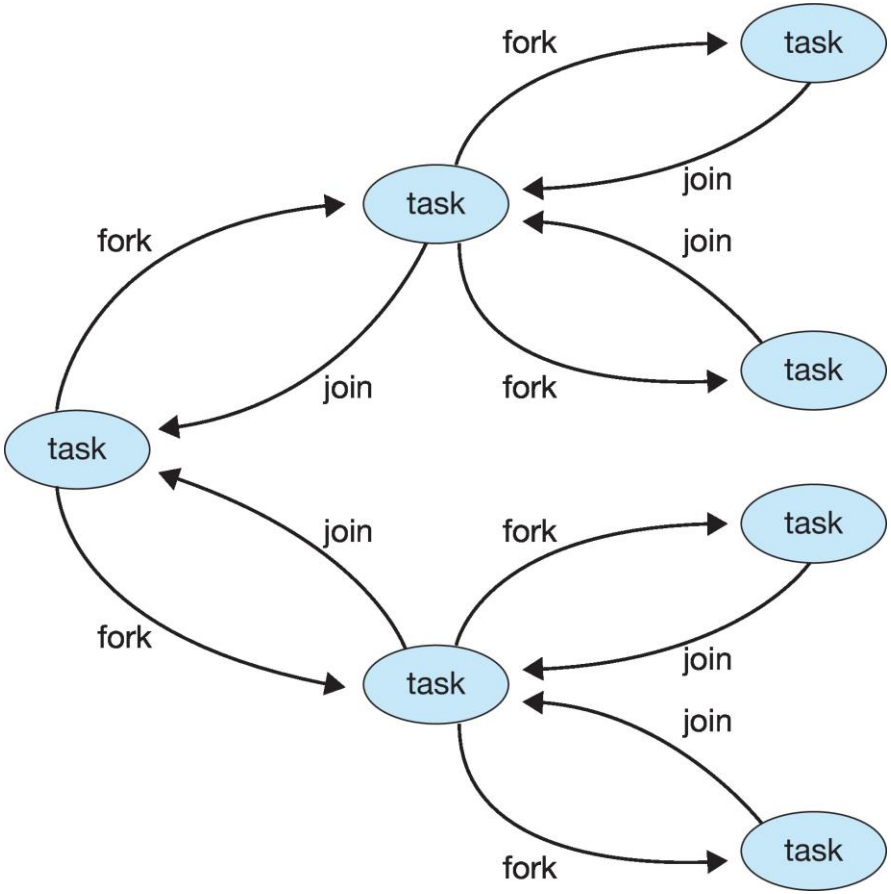
Fork-Join Parallelism

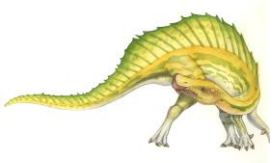
- General algorithm for fork-join strategy:

```
Task(problem)
   if problem is small enough
     solve the problem directly
   else
     subtask1 = fork(new Task(subset of problem)
     subtask2 = fork(new Task(subset of problem)

     result1 = join(subtask1)
     result2 = join(subtask2)

     return combined results
```



Fork-Join Parallelism



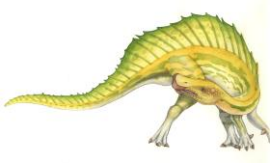


Threading Issues

- Semantics of **fork()** and **exec()** system calls
- Signal handling
  - Synchronous and asynchronous
- Thread cancellation of target thread
  - Asynchronous or deferred
- Thread-local storage

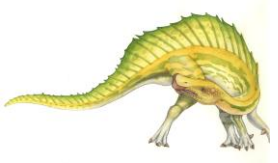
Semantics of fork() and exec()

- Does **`fork()`** duplicate only the calling thread or all threads?
  - Some UNIXes have two versions of fork
- **`exec()`** usually works as normal – replace the running process including all threads



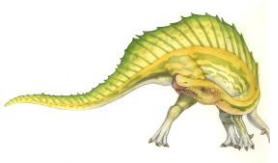
Signal Handling

- **Signals** are used in UNIX systems to notify a process that a particular event has occurred.
- A **signal handler** is used to process signals
  1. Signal is generated by particular event
  2. Signal is delivered to a process
  3. Signal is handled by one of two signal handlers:
     1. default
     2. user-defined
- Every signal has **default handler** that kernel runs when handling signal
  - **User-defined signal handler** can override default
  - For single-threaded, signal delivered to process



Signal Handling (Cont.)

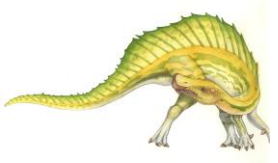
- Where should a signal be delivered for multi-threaded?
  - Deliver the signal to the thread to which the signal applies
  - Deliver the signal to every thread in the process
  - Deliver the signal to certain threads in the process
  - Assign a specific thread to receive all signals for the process

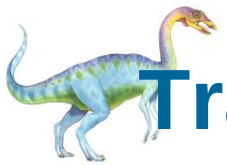


Tratarea semnalelor in Solaris

- fiecare thread are propria masca de semnale
- toate threadurile unui proces partajeaza handlerele de semnal
- cand SIG_IGN/SIG_DFL sunt setate, se aplica tuturor threadurilor procesului
- semnalele de tip *trap* (sincrone cu executia threadului, ex SIGFPE, SIGSEGV) executate exclusiv de threadul care le-a generat

  => mai multe threaduri pot genera si trata acelasi tip de semnal simultan
- semnalele de tip interupere (generate asincron cu executia threadului, ex SIGIO, SIGINT)
  - tratate de orice thread care nu mascheaza/blocheaza semnalul respectiv (pt. m.m. astfel de thread-uri, se alege unul dintre ele pt tratarea semnalului)
- pt semnal mascat in toate threadurile, se asteapta primul thread care deblocheaza tratarea semnalului respectiv





Tratarea semnalelor in Solaris (cont.)

- *thread_kill*: trimite un semnal catre un thread din acelasi proces
  - semnalul e de tip trap, tratat doar de threadul caruia ii este adresat
  - nu se pot trimite semnale unui **anume** thread dintr-un **alt** proces
- threadurile Solaris implementeaza un nou semnal, SIGWAITING (SIG_IGN implicit)
  - trimis procesului cand toate LWP-urile sunt blocate indefinit in asteptarea unui eveniment extern
  - poate fi folosit pt crearea de noi LWP-uri pt a evita blocarea intregului proces
  - idee similara cu scheduler activations, dar coarse-grain
    - nu merge pt short-term blocking, doar pt evenimente de tip page faults, filesystem I/O

Thread Cancellation

- Terminating a thread before it has finished
- Thread to be canceled is **target thread**
- Two general approaches:
  - **Asynchronous cancellation** terminates the target thread immediately
  - **Deferred cancellation** allows the target thread to periodically check if it should be cancelled
- Pthread code to create and cancel a thread:

```
pthread_t tid;

/* create the thread */
pthread_create(&tid, 0, worker, NULL);

  . . .

/* cancel the thread */
pthread_cancel(tid);

/* wait for the thread to terminate */
pthread_join(tid,NULL);
```

Thread Cancellation (Cont.)

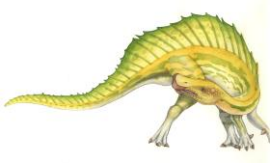
- Invoking thread cancellation requests cancellation, but actual cancellation depends on thread state

Mode	State	Type
Off	Disabled	–
Deferred	Enabled	Deferred
Asynchronous	Enabled	Asynchronous

- If thread has cancellation disabled, cancellation remains pending until thread enables it
- Default type is deferred
  - Cancellation only occurs when thread reaches **cancellation point**
    - i.e., **`pthread_testcancel()`**
    - Then **cleanup handler** is invoked
- On Linux systems, thread cancellation is handled through signals

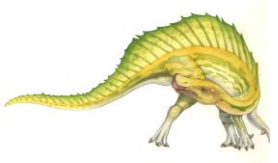
Thread-Local Storage

- **Thread-local storage** (**TLS**) allows each thread to have its own copy of data
- Useful when you do not have control over the thread creation process (i.e., when using a thread pool)
- Different from local variables
  - Local variables visible only during single function invocation
  - TLS visible across function invocations
- Similar to `static` data
  - TLS is unique to each thread



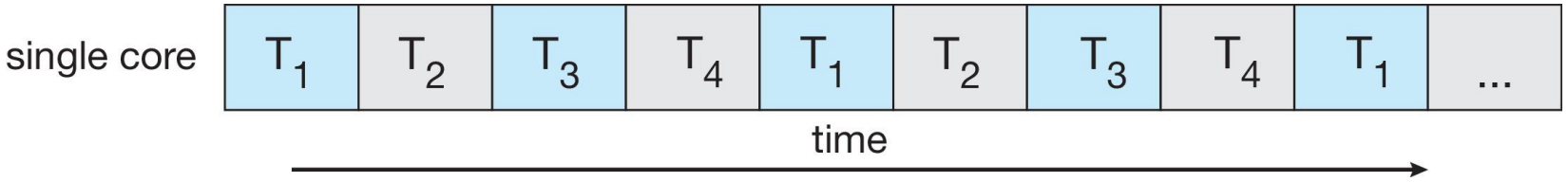
Multicore Programming

- **Multicore** or **multiprocessor** systems puts pressure on programmers, challenges include:
  - **Dividing activities**
  - **Balance**
  - **Data splitting**
  - **Data dependency**
  - **Testing and debugging**
- ***Parallelism*** implies a system can perform more than one task simultaneously
- ***Concurrency*** supports more than one task making progress
  - Single processor / core, scheduler providing concurrency

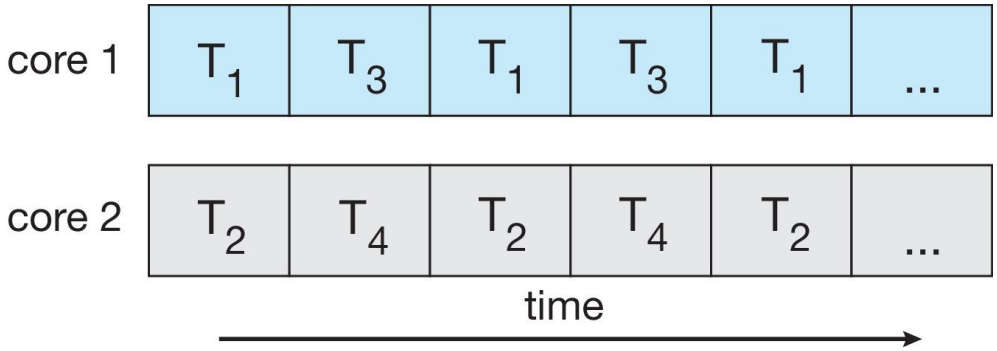


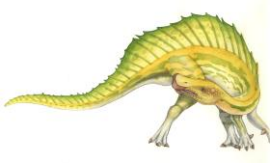
Concurrency vs. Parallelism

- **Concurrent execution on single-core system:**



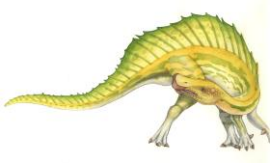
- **Parallelism on a multi-core system:**



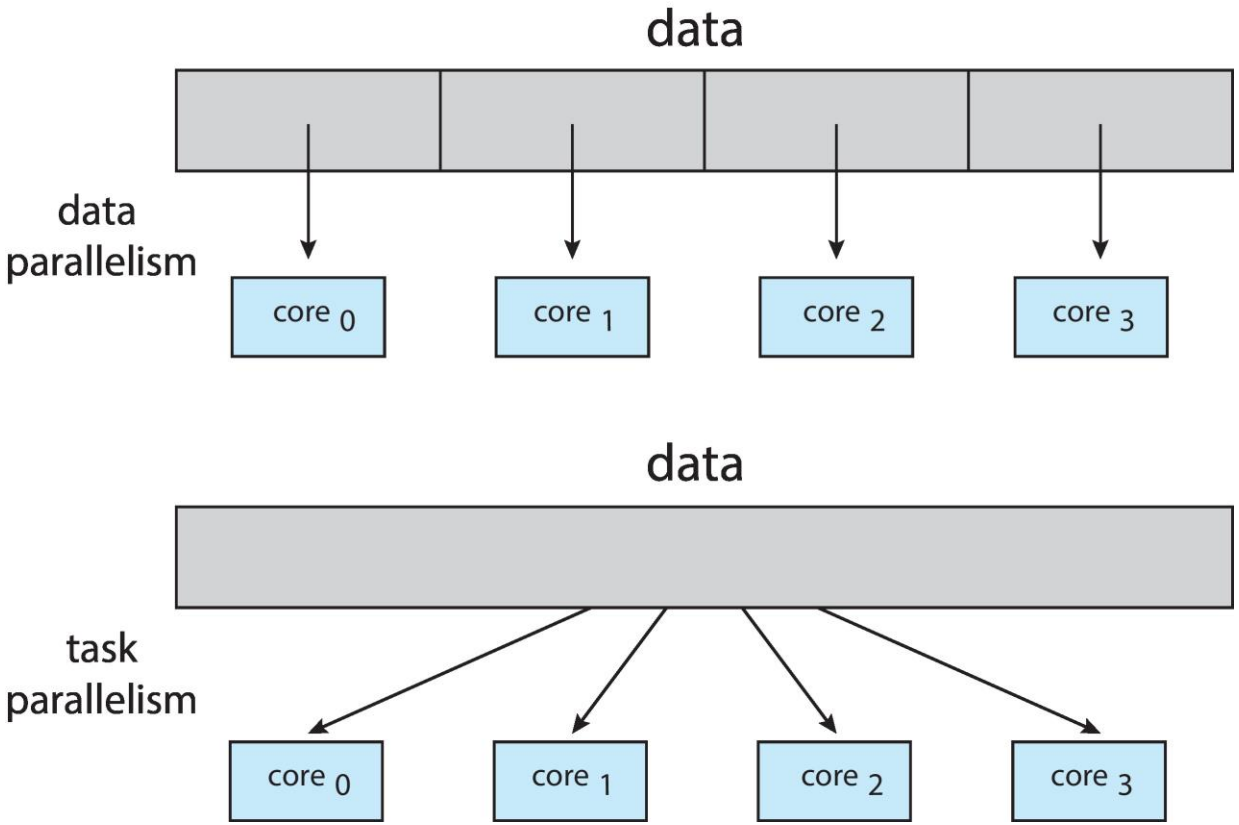


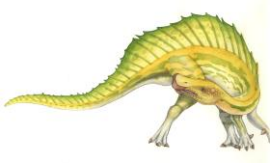
Multicore Programming

- Types of parallelism
  - **Data parallelism** – distributes subsets of the same data across multiple cores, same operation on each
  - **Task parallelism** – distributing threads across cores, each thread performing unique operation



Data and Task Parallelism





Amdahl's Law

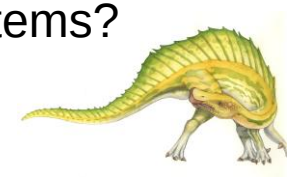
- Identifies performance gains from adding additional cores to an application that has both serial and parallel components
- *S* is serial portion
- *N* processing cores

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

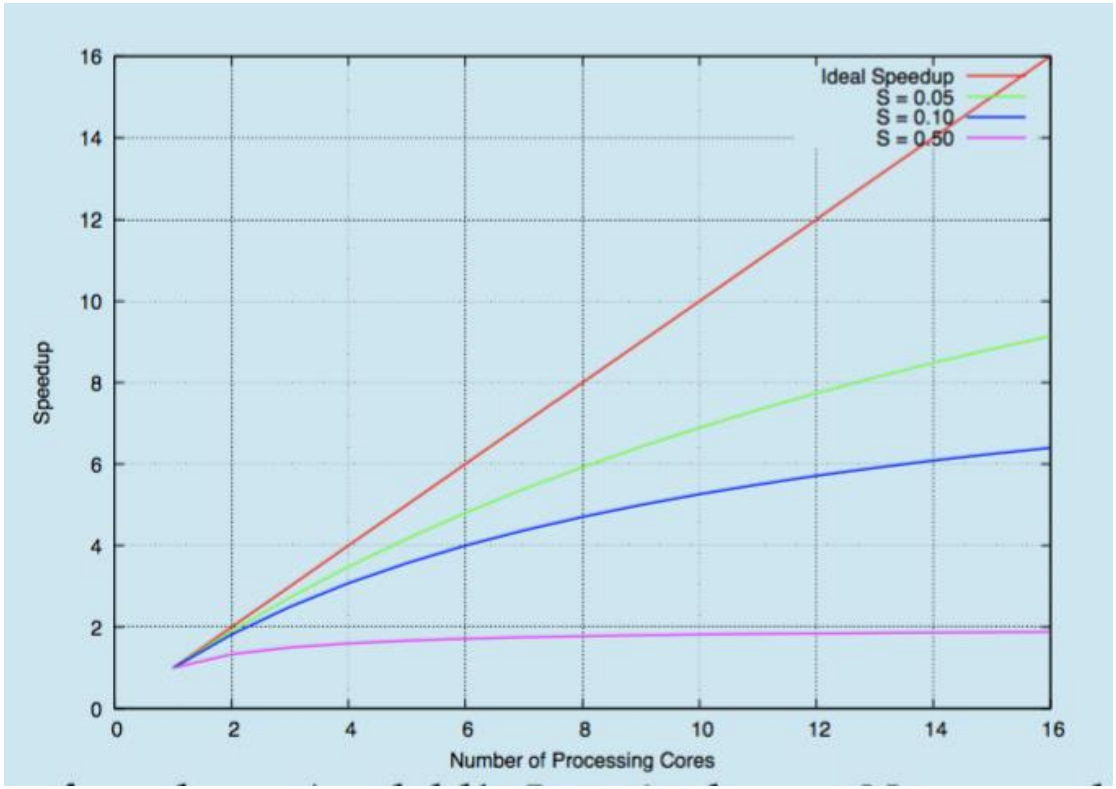
- That is, if application is 75% parallel / 25% serial, moving from 1 to 2 cores results in speedup of 1.6 times
- As *N* approaches infinity, speedup approaches 1 / *S*

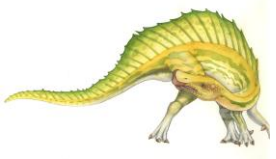
Serial portion of an application has disproportionate effect on performance gained by adding additional cores

- But does the law take into account contemporary multicore systems?



Amdahl's Law


Ideal Speedup
S = 0.05
S = 0.10
S = 0.50
Speedup
Number of Processing Cores
0
2
4
6
8
10
12
14
16



End of Chapter 4